

Ingeniería de Software II

Metodologías Clásicas (predictivas) y Ágiles

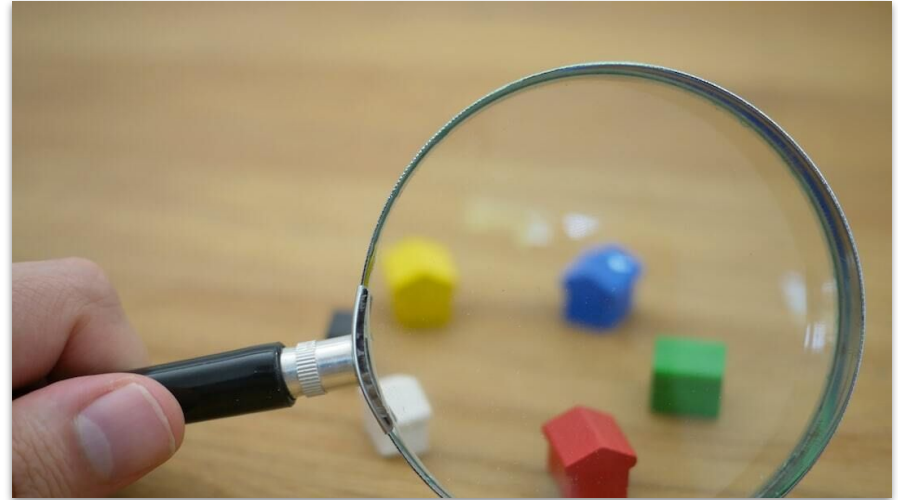
Metodologías de Proyectos

- **Concepto:** La gestión de proyectos de software es una disciplina que aplica principios y técnicas de administración para planificar, ejecutar y supervisar proyectos de desarrollo de software.
- **Importancia:** Garantiza que los proyectos se completen a tiempo, dentro del presupuesto y con la calidad requerida, minimizando riesgos y maximizando la eficiencia.
- **Objetivos:** Definir y controlar el alcance, tiempo, costo y calidad del proyecto para satisfacer las necesidades del cliente y los requisitos del negocio.

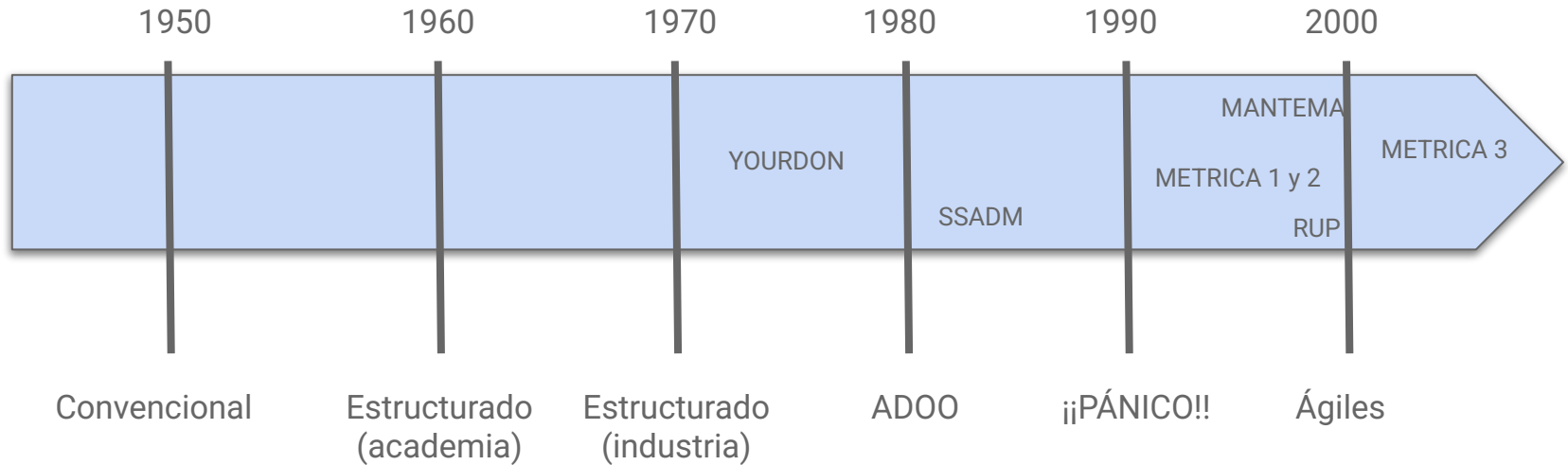


Metodologías de Desarrollo: Criterios de Selección

- **Tamaño del Proyecto:** Proyectos pequeños pueden beneficiarse de metodologías ágiles, mientras que proyectos grandes y complejos pueden requerir enfoques más estructurados.
- **Equipo y Habilidades:** La experiencia y habilidades del equipo influyen en la elección de la metodología. Equipos experimentados pueden manejar metodologías ágiles más flexibles.
- **Requisitos del Cliente:** La claridad y estabilidad de los requisitos del cliente determinan la adaptabilidad necesaria en la metodología seleccionada.



Metodologías de Proyectos

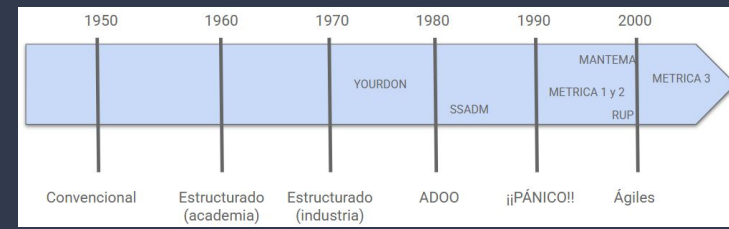


Metodología Estructurada

- **Definición:** Es una metodología que divide el desarrollo en fases secuenciales y bien definidas, como análisis, diseño, implementación y pruebas.
- **Ventajas:** Proporciona un marco claro y estructurado, facilita la gestión de proyectos grandes y la estimación de tiempos y costos.
- **Desventajas:** Poca flexibilidad para cambios durante el desarrollo y puede no ser adecuada para proyectos con requisitos no bien definidos desde el inicio.



Metodología Estructurada



Metodología de Yourdon/Constantine

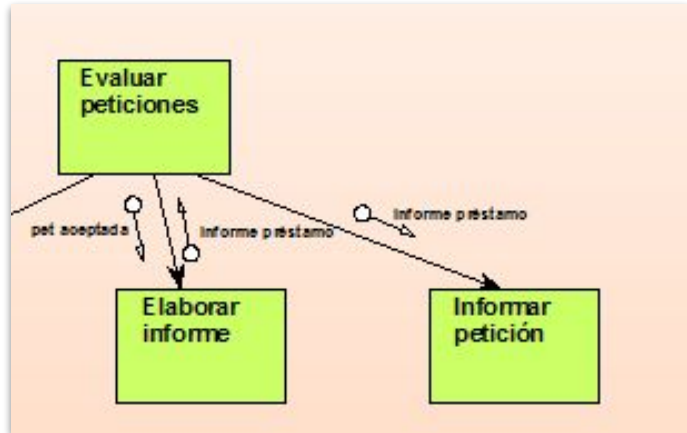
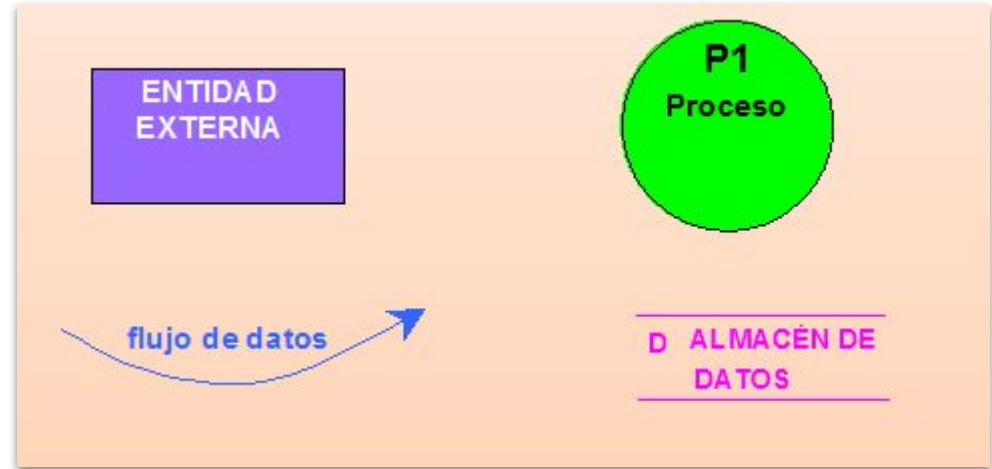
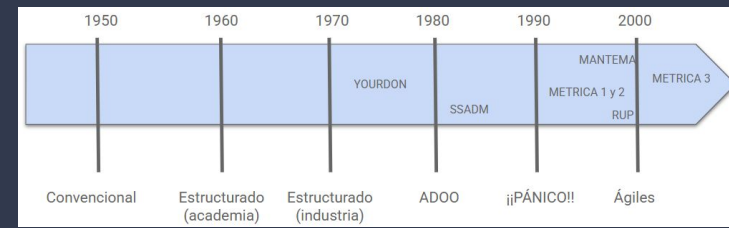


Diagrama de estructura

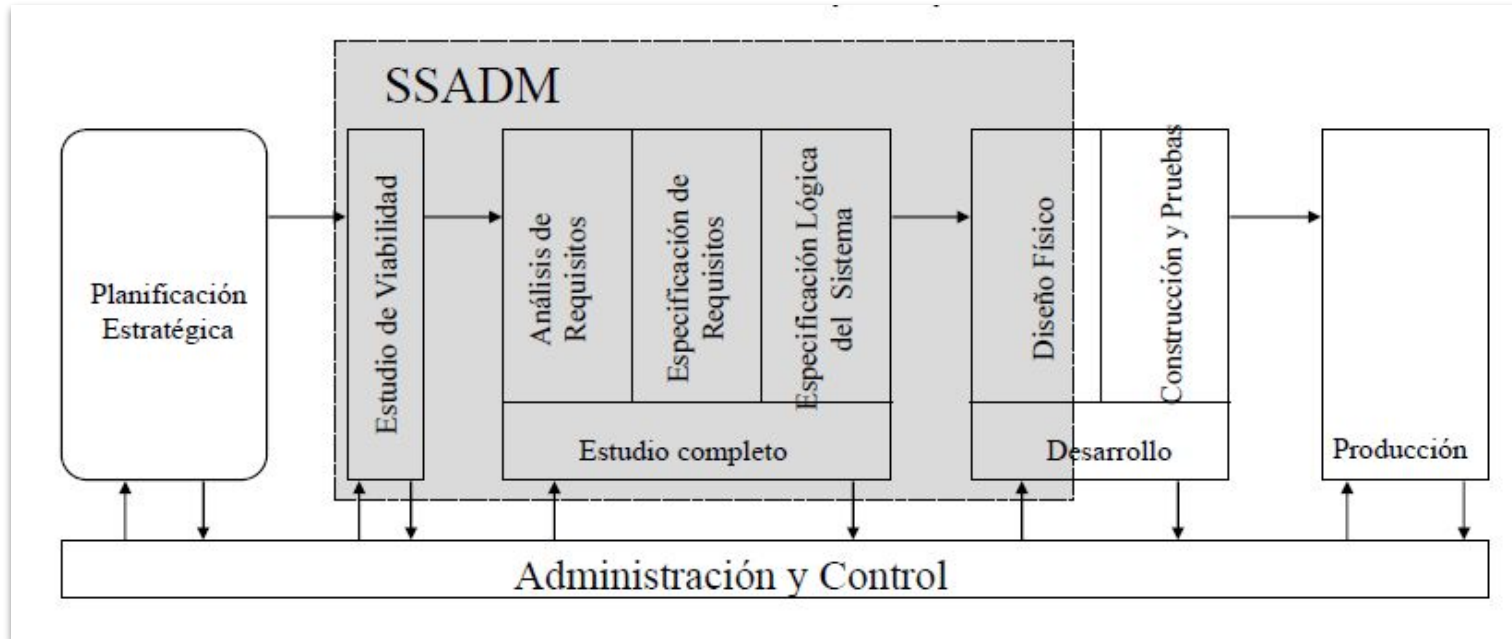


DFD

Metodología Estructurada



Structured Systems Analysis and Design Method - SSADM

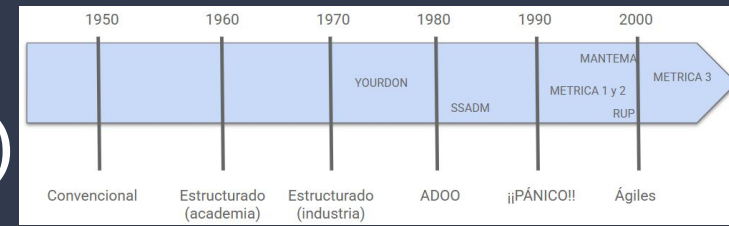


Metodología Orientada a Objetos

- **Definición:** Se basa en el concepto de 'objetos' que representan entidades del mundo real con atributos y comportamientos, facilitando la reutilización y modularidad del código.
- **Ventajas:** Promueve la reutilización de código, facilita el mantenimiento y la escalabilidad, y mejora la gestión de la complejidad.
- **Desventajas:** Puede ser más compleja de aprender y aplicar, y requiere una buena planificación y diseño para aprovechar sus beneficios.

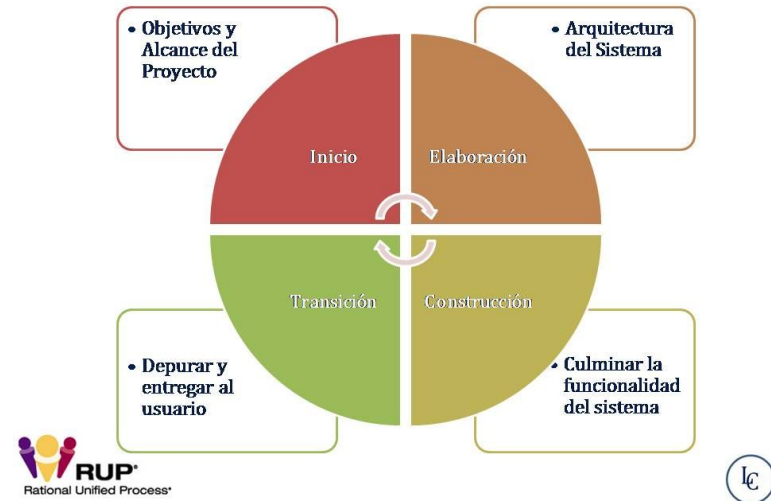


RUP (Rational Unified Process)

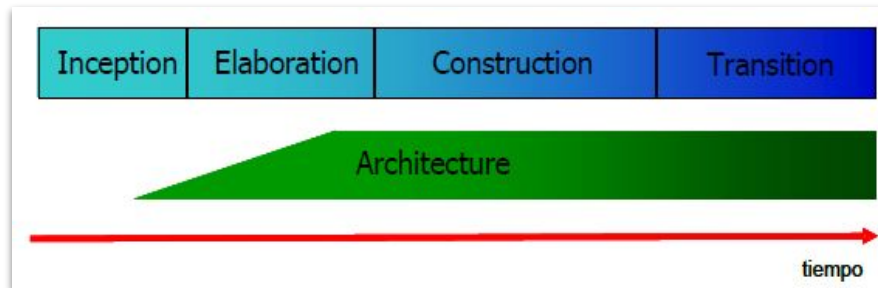
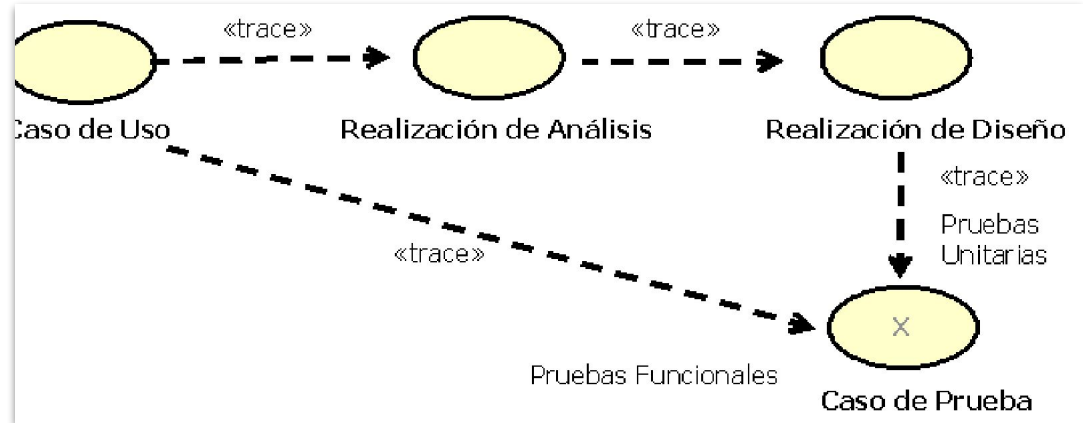
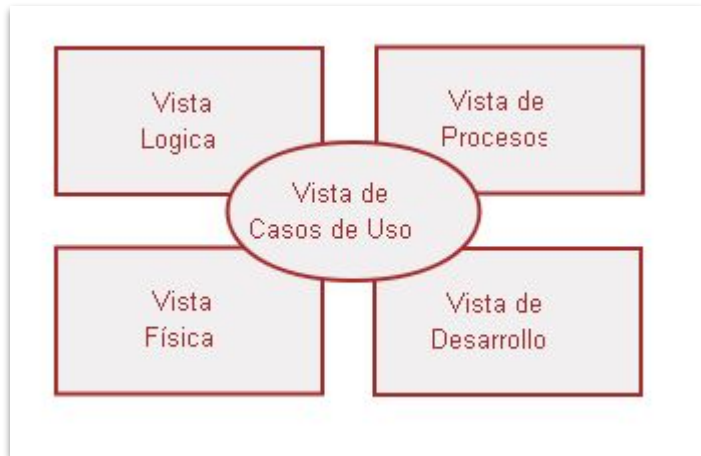


- **Definición:** Modelo de desarrollo de software iterativo e incremental que busca mejorar la productividad y la calidad de los proyectos de software. Fue desarrollado por la empresa Rational Software, que más tarde fue adquirida por IBM.
- Estructurado en torno a cuatro fases principales: inicio, elaboración, construcción y transición, cada una con objetivos y actividades específicas..
- Utiliza UML como notación estándar

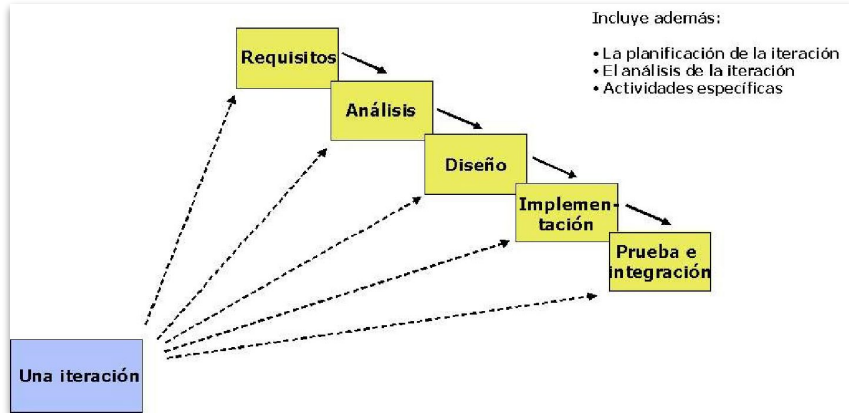
Fases del Método RUP



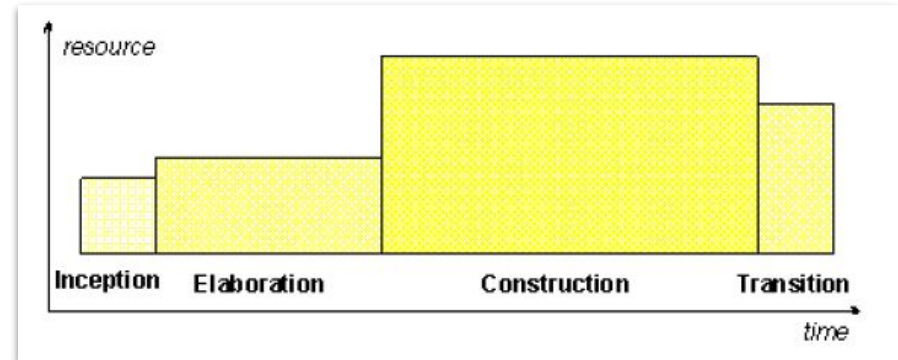
RUP (Rational Unified Process)



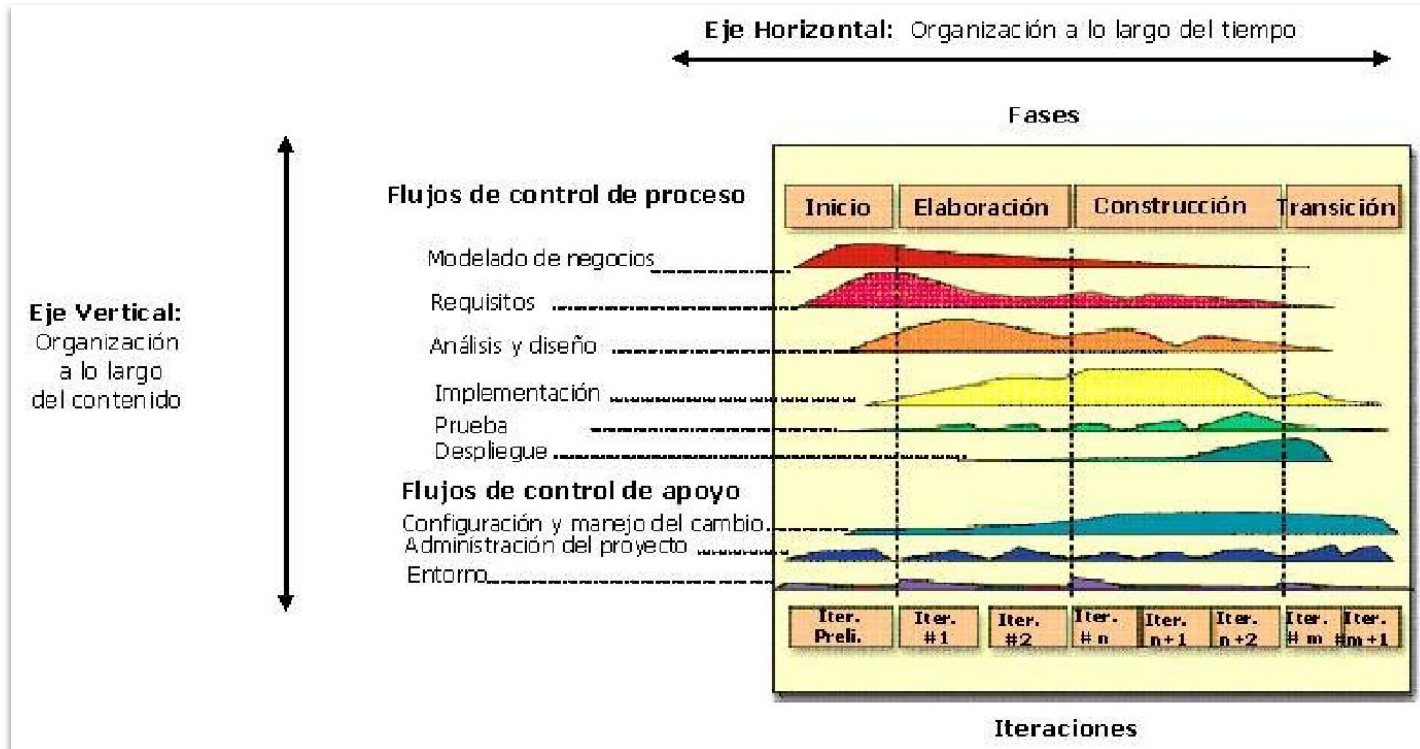
RUP (Rational Unified Process)



	Inicio	Elaboración	Construcción	Transición
Esfuerzo	5 %	20 %	65 %	10%
Tiempo Dedicado	10 %	30 %	50 %	10%



RUP (Rational Unified Process)



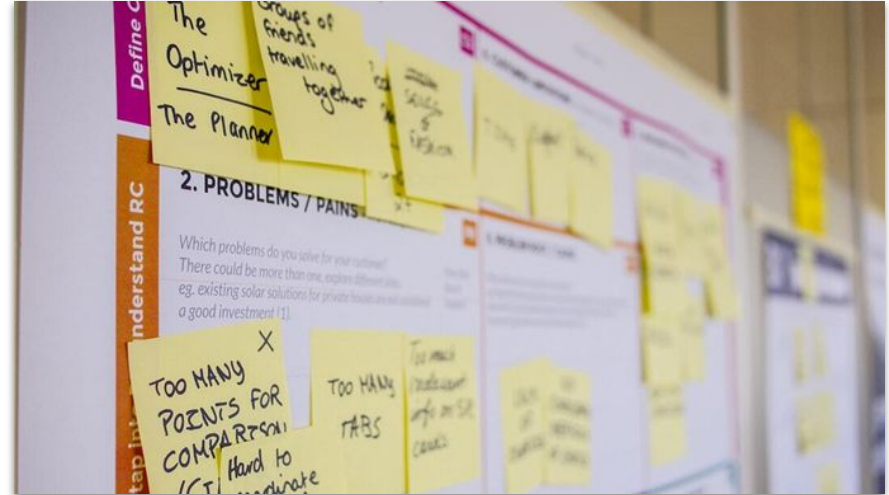
Metodología PRINCE2

- **Origen y Definición:** PRINCE2 (PRojects IN Controlled Environments) es una metodología de gestión de proyectos desarrollada en el Reino Unido, ampliamente adoptada a nivel mundial. Su enfoque se centra en la organización, gestión y control de proyectos.
- **Principales Características:** Define roles y responsabilidades claros, procesos estructurados y la importancia de la justificación continua del negocio durante todo el ciclo de vida del proyecto.



Metodología PRINCE2

- **Inicio de Proyecto:** Definición de los objetivos del proyecto, el equipo y la organización. Se crea el acta de constitución del proyecto.
- **Dirección de Proyecto:** Tomar decisiones estratégicas, aprobar fases y resolver problemas escalados. Se mantiene la justificación del negocio.
- **Control de Fase:** Monitoreo y control de la ejecución de una fase del proyecto. Se gestionan riesgos y problemas.



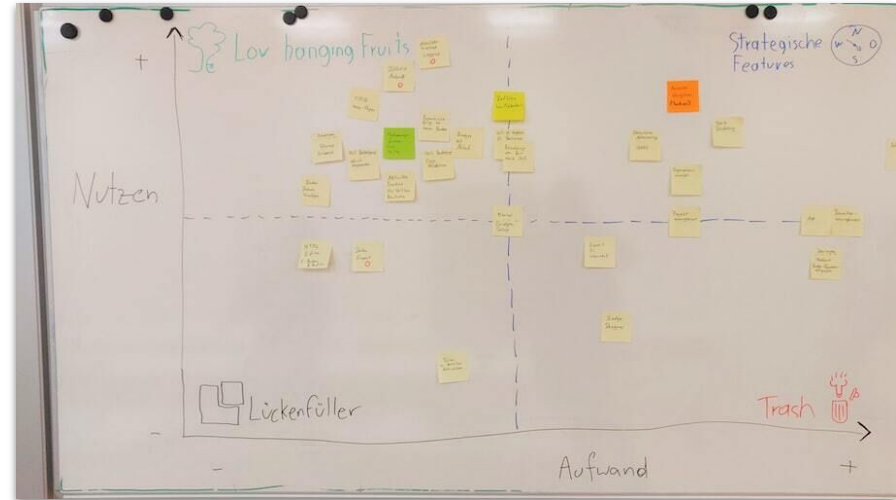
Metodología PRINCE2

- **Justificación Continua del Negocio:** *Asegura que el proyecto sigue siendo viable y rentable a lo largo de su ciclo de vida.*
- **Gestión de Riesgos:** Identificación, evaluación y control de riesgos que puedan afectar al proyecto.
- **Calidad:** Definición y control de los estándares de calidad requeridos para cumplir con los objetivos del proyecto.



Introducción a las Metodologías Ágiles

- **Definición:** Las metodologías ágiles son enfoques de desarrollo de software que enfatizan la flexibilidad, la colaboración y la entrega continua de valor al cliente.
- **Principios Básicos:** Incluyen la adaptación al cambio, la entrega de incrementos funcionales de software, y la colaboración cercana con el cliente y entre los miembros del equipo.
- **Manifiesto Ágil:** Un documento fundacional que establece valores y principios para guiar el desarrollo ágil, como la prioridad en las personas y las interacciones sobre los procesos y herramientas.



Manifiesto Ágil

Manifiesto por el Desarrollo Ágil de Software

Estamos descubriendo formas mejores de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de este trabajo hemos aprendido a valorar:

Individuos e interacciones sobre procesos y herramientas

Software funcionando sobre documentación extensiva

Colaboración con el cliente sobre negociación contractual

Respuesta ante el cambio sobre seguir un plan

Esto es, aunque valoramos los elementos de la derecha, valoramos más los de la izquierda.

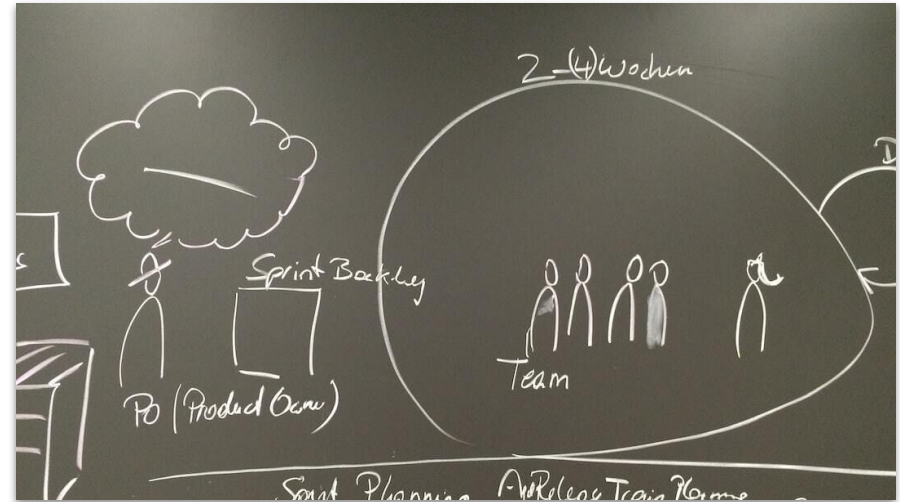
Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

Firmado en febrero de 2001

- Kent Beck: XP
- Mike Beedle: pionero de Scrum
- Arie van Bennekum: Dynamic Systems Development Method
- Alistair Cockburn: autor prolífico y metodología “Crystal Clear”
- Ward Cunningham: XP + formato Wiki
- Martin Fowler: refactorización de código y patrones de diseño
- James Grenning
- Jim Highsmith
- Andrew Hunt: coautor "The Pragmatic Programmer"
- Ron Jeffries: XP
- Jon Kern
- Brian Marick
- Robert C. Martin: autor “Code Clean”
- Steve Mellor
- Ken Schwaber: Scrum
- Jeff Sutherland: Scrum
- Dave Thomas: coautor "The Pragmatic Programmer"

Metodologías Ágiles y sus Variantes

- **Principios Ágiles:** Se enfocan en la entrega rápida y continua de software funcional, adaptándose a los cambios y colaborando estrechamente con el cliente.
- **Scrum:** Una de las metodologías ágiles más populares, que utiliza sprints cortos y reuniones diarias para gestionar y completar tareas.
- **XP (Extreme Programming):** Enfocada en la calidad del software y la capacidad de respuesta a los requisitos cambiantes, utilizando prácticas como la programación en pareja y las pruebas continuas.



Metodologías Ágiles y sus Variantes

- **Unified Process (UP):** Es una metodología iterativa y incremental para el desarrollo de software que se centra en la arquitectura del sistema y la gestión de riesgos.
- **EssUP (Essential Unified Process):** Es una versión simplificada y esencial del UP, diseñada para ser más fácil de adoptar y aplicar en diferentes tipos de proyectos (Ivar Jacobson).
- **Ventajas:** Fomenta una comprensión clara de los requisitos y la arquitectura, y permite la gestión efectiva de cambios y riesgos.



XP (Extreme Programming)

- **Definición:** XP es una metodología ágil que se enfoca en mejorar la calidad del software y la capacidad de respuesta a los cambios a través de ciclos de desarrollo cortos y feedback continuo.
- **Prácticas Clave:** Incluye prácticas como la programación en pareja, desarrollo dirigido por pruebas (TDD), integración continua y refactorización constante.
- **Ventajas:** Promueve la comunicación constante, la calidad del código y la flexibilidad para adaptarse a cambios en los requisitos del cliente.



Los 5 Valores de XP

1. Comunicación
2. Simplicidad
3. Feedback
4. Respeto
5. Coraje



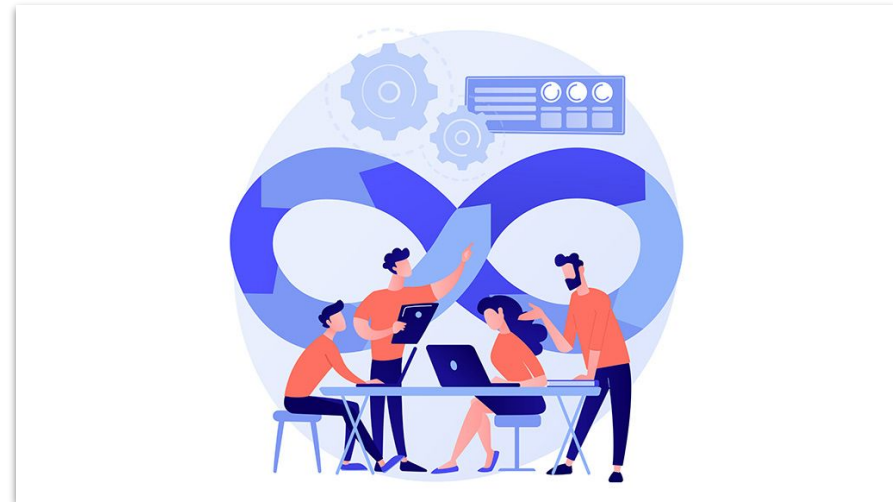
Las 12 prácticas de XP

1. Cliente In-Situ
2. Semana de 40 Horas
3. Uso de Metáforas
4. Diseño Simple
5. Refactoring
6. Pair Programming
7. Entregas Cortas
8. Pruebas
9. Estándares de Código
10. Propiedad Colectiva
11. Integración Continua
12. Juego de Planificación



Lean

- **Definición:** Lean es una metodología que busca maximizar el valor para el cliente y minimizar el desperdicio a través de la mejora continua y la eficiencia en los procesos.
- **Principios:** Incluyen la identificación y eliminación del desperdicio, la creación de flujo continuo, y la mejora continua mediante ciclos de retroalimentación.
- **Aplicación en Software:** En el desarrollo de software, Lean se enfoca en entregar valor rápidamente, reducir tiempos de espera y evitar tareas que no aporten directamente al objetivo del proyecto.



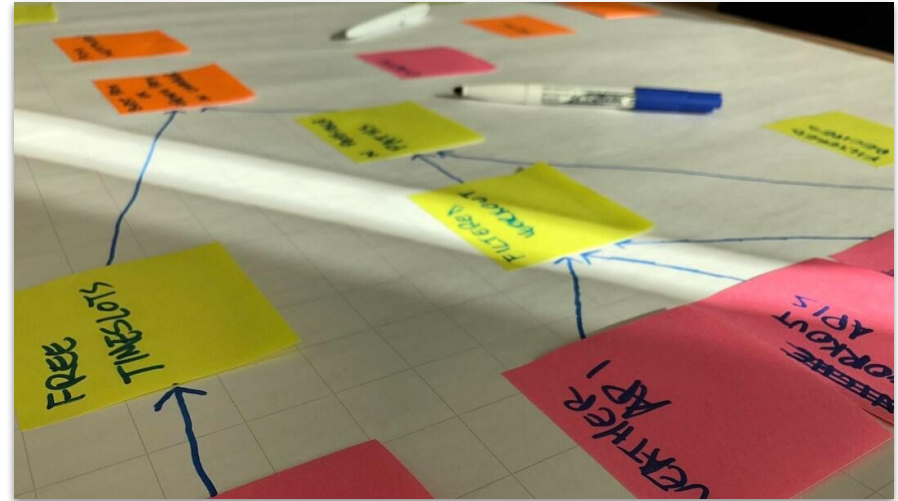
Test-Driven Development (TDD)

- **Definición:** TDD es una metodología de desarrollo de software en la que las pruebas se escriben antes del código, guiando el desarrollo y asegurando que el software cumpla con los requisitos especificados.
- **Ciclo TDD:** Consta de tres etapas: escribir una prueba que falle, escribir el código mínimo necesario para pasar la prueba, y refactorizar el código para mejorar su estructura manteniendo las pruebas verdes.
- **Ventajas:** Ayuda a prevenir errores, facilita el diseño modular, mejora la calidad del código y proporciona documentación actualizada del comportamiento esperado del software.

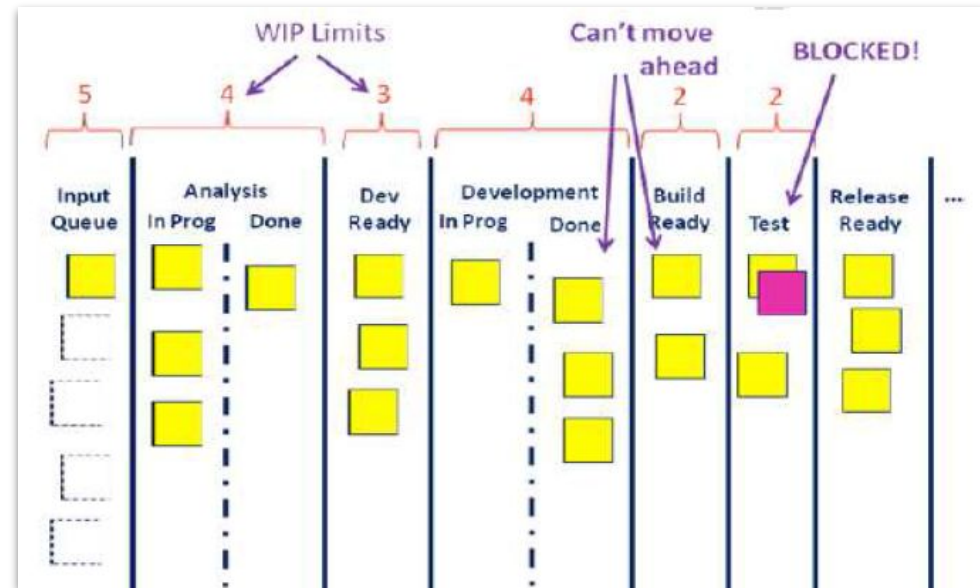
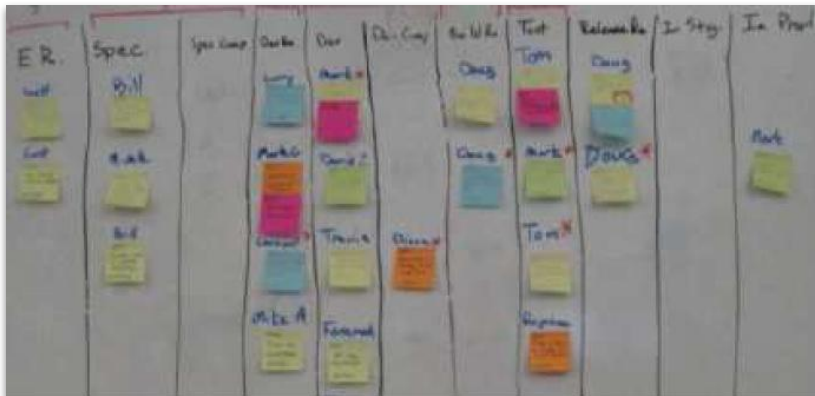
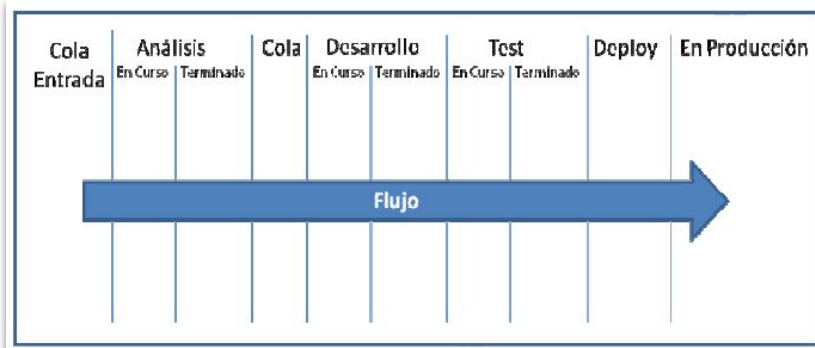


Kanban

- **Definición:** Kanban es una metodología ágil que utiliza tarjetas visuales en un tablero para gestionar y optimizar el flujo de trabajo, permitiendo ver el estado de las tareas en tiempo real.
- **Principios:** Se basa en la visualización del trabajo, la limitación del trabajo en progreso, la gestión del flujo, y la mejora continua.
- **Ventajas:** Facilita la identificación de cuellos de botella, mejora la comunicación del equipo y aumenta la flexibilidad para adaptarse a cambios en las prioridades.

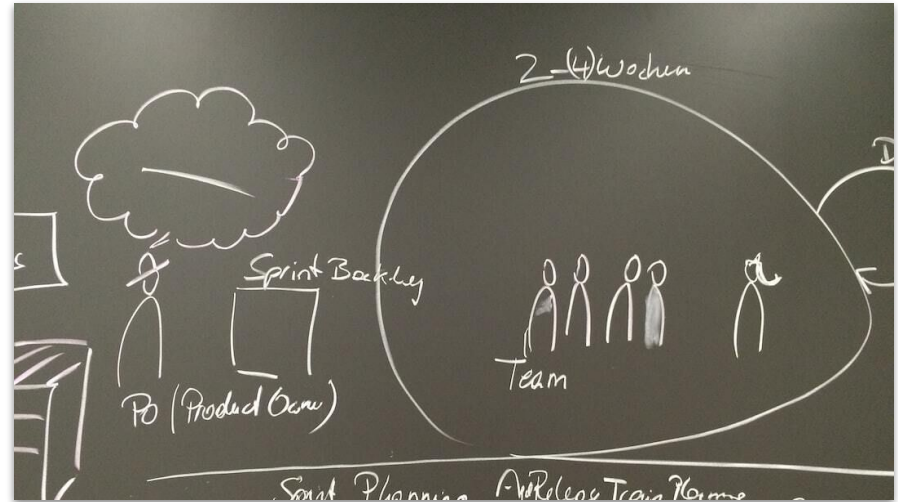


Kanban



Scrum

- **Definición:** Scrum es un marco de trabajo ágil que facilita el desarrollo y la entrega de productos complejos a través de iteraciones cortas llamadas sprints.
- **Roles Clave:** Incluye tres roles principales: el Product Owner, el Scrum Master y el equipo de desarrollo, cada uno con responsabilidades específicas.
- **Eventos (“ceremonias”):** Los eventos clave en Scrum son el Sprint Planning, Daily Scrum, Sprint Review y Sprint Retrospective, que ayudan a planificar, ejecutar y revisar el trabajo realizado.



Scrum

Jeff Sutherland → proceso de Scrum → 1993

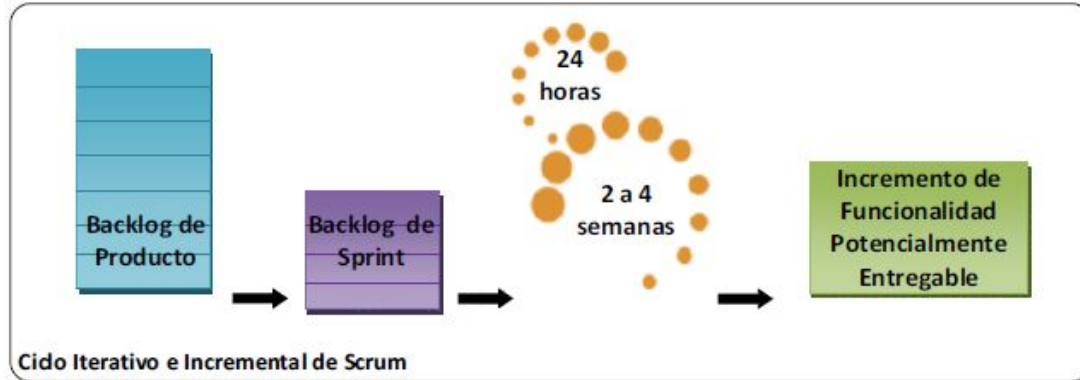
Componentes:

- Organización → Sprints
- Roles
- Artefactos
- Eventos (“ceremonias”)



Sprints

- Sprint → **iteración**
- Duración **fija** para todos los sprints: una a cuatro semanas
- Se ejecutan uno tras el otro respetando esta duración, sin tiempo muerto entre el sprint que termina y el sprint que empieza.
- Objetivo: **transformar un conjunto acordado de ítems** del Backlog de Producto en un Incremento de Funcionalidad de Producto Potencialmente Deployable



Roles 1/3

Dueño del Producto

- Voz del cliente
- Establece una visión del producto a desarrollar
- Prioriza continuamente los requisitos para alcanzar dicha visión
- Es parte del equipo
- Responsable de canalizar las necesidades del negocio para el producto a desarrollar, maximizar el valor para el negocio respecto a la inversión, inspeccionar y adaptar el producto
- Autoridad final
- Asegurarse que el equipo entiende la visión del producto y sus distintos requisitos

Roles 2/3

Scrum Master

- Facilitador y agente de cambio para la aplicación adecuada de Scrum
- Coaching al equipo para ser más productivo y a generar productos de mejor calidad
- Eliminar los impedimentos que frenan el trabajo del equipo
- Proteger al equipo de potenciales interferencias proveniente de la organización

Roles 3/3

Equipo

- Grupo auto-organizados, multidisciplinario (programación, pruebas, análisis funcional, arquitectura, diseño de interfaz gráfica o diseño de base de datos) y con autonomía que desarrolla el producto.
- Responsabilidad: transformar el Backlog de Producto en incrementos de funcionalidad potencialmente entregable en cada sprint.
- Promover el aprendizaje y perfeccionamiento de todos los miembros del equipo de todas las tareas involucradas en el desarrollo.
- Auto-gestión: de ninguna manera el Scrum Master o el Dueño de Producto le explica al equipo como transformar el Backlog de Producto en incrementos de funcionalidad potencialmente entregable.
- El equipo tiene autonomía para definir cómo se debe cumplir este objetivo y cómo organizar el trabajo de todos sus miembros para hacerlo de la mejor manera posible, pidiendo claridad y apoyo del Dueño de Producto cuando es necesario.
- Tamaño recomendado: 7 persona +/- 2 (sin Dueño de Producto y Scrum Master)
 - Cuando son menos de cinco, hay menos interacciones y como consecuencia hay menos productividad.
 - Si hay más de nueve personas, se requiere demasiada coordinación y se puede generar demasiada complejidad en las actividades grupales de Scrum.

Artefactos 1/4

Backlog de Producto

- Definición: es una lista única, pública y dinámica priorizada de requerimientos
- Algunos de estos requerimientos tienen una estimación de alto nivel de esfuerzo o complejidad.
- Representa el "qué" esperado del producto, sin preocuparse por el "cómo"

Historias de Usuario:

- Forma de escribir un ítem del Backlog de Producto que se enfoca en la necesidad de los involucrados de entender como la funcionalidad se va a usar.
- Permite al equipo entender la perspectiva del usuario antes de elegir una forma de implementarlo.
- Se escribe de la forma siguiente:
 - Como [un usuario], puedo [una funcionalidad] para [un beneficio].
 - Por ejemplo: "Como usuario no registrado, puedo reservar consultar los vuelos precios de los vuelos para calcular el presupuesto de mi viaje ".
- Épica: historia de usuario que no puede ser entregada tal y como se ha definido dentro de una sola iteración, o que es suficientemente grande como para ser partida en historias de usuario más pequeñas.

Artefactos 2/4

Backlog de Sprint

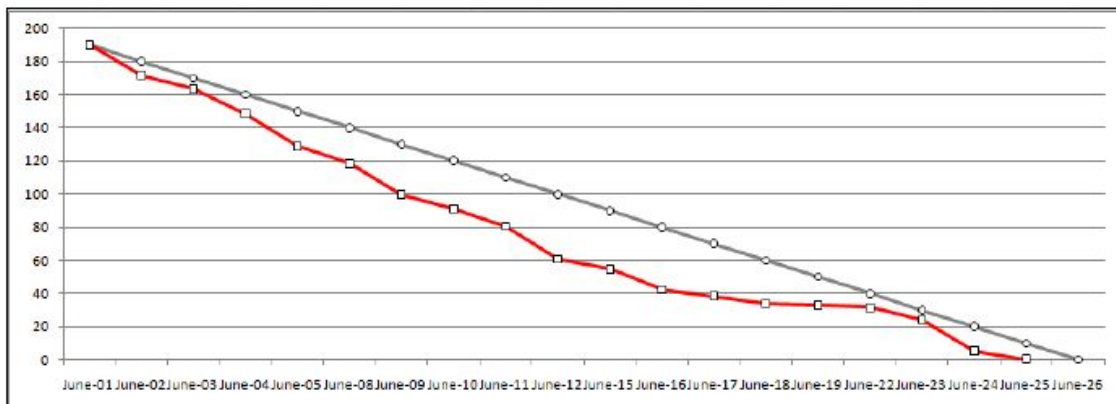
- **Definición:** conjunto reducido negociado de ítems del Backlog de Producto que el equipo se compromete a completar durante el tiempo alocado al Sprint.
- Cada ítem incluido en el sprint es **dividido** en tareas detalladas que el equipo va a completar, con una estimación de duración que no supere un día de esfuerzo.
- Es **actualizado diariamente** por el equipo, y muestra en cada momento las tareas pendientes, en curso y terminadas para completar los ítems del sprint, así como una estimación del esfuerzo pendiente de cada tarea no terminada y a quién está asignada la tarea.
- Se suele utilizar un **Tablero de Sprint** como forma de visualizar el Backlog de Sprint, o algún otro soporte herramental que permite darle visibilidad.



Artefactos 3/4

Diagrama de Burndown

- Forma gráfica de mostrar el avance de un sprint



- En este ejemplo vemos un sprint de 20 días de duración, con un esfuerzo restante estimado inicial de 190 horas.

Artefactos 4/4

Estimaciones de Alto-Nivel

- El Equipo determina una estimación de alto nivel de cada uno de los elementos del Backlog de Producto presentados
- Para eso puede ser necesario tomar algunas decisiones iniciales de diseño y hacer preguntas adicionales al Dueño de Producto para asegurar un buen entendimiento del ítem.

Técnicas de estimación

- Planning Poker
- T-Shirt Sizing
- Wideband Delphi

Ceremonias 1/4

Planificación

- ¿Cuándo?: Al iniciar el sprint, es la primera actividad de cada sprint.
- Participantes: Dueño de Producto + Scrum Master + Equipo
- Objetivos: que el equipo entienda los ítems más prioritarios, que el equipo y Dueño de Producto acuerden el Sprint Backlog, que el equipo defina las tareas requeridas para construir las funcionalidades
- Duración: entre dos y ocho horas.
- Artefactos Involucrados
 - Backlog de Producto
 - Backlog de Sprint
- Dinámica
 - Entendimiento y Selección de Ítems del Backlog de Producto
 - Definición del Backlog de Sprint

Ceremonias 2/4

Reunión diaria

- ¿Cuándo?: Una vez por día, en un horario fijo. Se recomienda la primera hora del día de trabajo.
- Participantes: Scrum Master + Equipo + Dueño de Producto (opcional)
- Objetivos: Que los miembros del Equipo de Desarrollo sincronicen sus tareas, comuniquen sus avances y reporten al Scrum Master cualquier impedimento que pueda frenar o bloquear su trabajo.
- Duración: Menor a quince minutos
- Artefactos Involucrados:
 - Backlog de Sprint
- Dinámica: se hacen las siguientes preguntas a cada miembro del equipo uno por uno:
 - ¿Qué hiciste desde la última reunión diaria?
 - ¿Qué vas a hacer hasta la próxima reunión diaria?
 - ¿Hay algún obstáculo que frena o bloquea tu trabajo?
- Finalmente el Scrum Master suele actualizar el Diagrama de Burndown

Ceremonias 3/4

Revisión

- ¿Cuándo? El último día del sprint.
- Participantes: Scrum Master + Equipo + Dueño de Producto + Otros Involucrados/Interesados
- Objetivos
 - Que los interesados en el producto puedan visualizar y/o experimentar las funcionalidades construidas durante el sprint.
 - Que el Dueño de Producto pueda aceptar o rechazar dichas funcionalidades.
 - Que los interesados en el producto puedan elaborar eventuales mejoras al producto en base a lo visto.
- Duración: una o dos horas para exponer todas las funcionalidades desarrolladas.
- Artefactos Involucrados:
 - Incremento de Funcionalidad Potencialmente Deployable
- Dinámica
 - Breve presentación de los ítems del Backlog de Producto comprometidos
 - Demostración con el software andando de las funcionalidades desarrolladas
 - El Dueño de Producto puede pedir pasar a producción algunas funcionalidades terminadas
 - El Scrum Master anuncia la fecha y lugar de la próxima reunión de Revisión

Ceremonias 4/4

Retrospectiva

- ¿Cuándo? Al finalizar el sprint, es la última actividad del sprint.
- Participantes: Scrum Master + Equipo + Dueño de Producto (opcional)
- Objetivos: Inspeccionar los procesos de trabajo y las interacciones que ocurrieron en el último sprint para detectar fortalezas del equipo y oportunidades de mejora. Se busca acordar unas pocas acciones concretas de mejora a implementar en el próximo sprint para hacerlo más productivo.
- Duración: entre una y tres horas (varía según la cantidad de personas y la duración del sprint).
- Artefactos Involucrados:
 - No aplica.
- Dinámica: Actividades grupales facilitadas por el Scrum Master los puntos siguientes:
 - ¿Qué mecanismos, procedimientos o interacciones han funcionado bien en este sprint?
 - ¿Qué mecanismos, procedimientos o interacciones hay que mejorar?

OPTATIVA: Refinamiento del Backlog

- ¿Cuándo? Durante el sprint
- Objetivo: Revisión, ordenamiento, unión de HU, reestimación, etc.

Scrum

Palabra de Sutherland

- El tiempo es finito. Trátalo como tal: divide tu trabajo en lo que puedes hacer en un periodo corto, fijo y regular de entre una y cuatro semanas.
- Muestra o muerte. Al final de cada sprint, ten algo terminado y que pueda usarse.
- Tira tus tarjetas de presentación. Los epítetos de roles son indicadores de estatus especializado. Que te conozcan por lo que haces, no por tu profesión.
- Todos saben todo. La exhaustividad de la comunicación acelera el trabajo.
- Una reunión al día. Cuando se trata de revisiones de equipo, una vez al día basta. Reúnanse quince minutos, vean qué pueden hacer para avanzar más rápido y llévenlo a la práctica.

Definition of Ready (DoR)

Definición de “listo”

- Sirve para evaluar el trabajo **antes de que el equipo comience a desarrollarlo**.
- Define una tarea, historia de usuario o punto de historia, y permite actuar de inmediato si se trabaja con Scrum.
- A qué clientes se dirige: sus motivaciones, problemas y necesidades.
- Las tareas requeridas: si aportan valor para el negocio y el usuario, y si son claras y factibles.
- Los requisitos técnicos: recursos disponibles, enfoque o solución técnica y capacidad de prueba.
- Las estimaciones de tiempo: plazos para completar el trabajo y acuerdo con los interesados.

Definition of Done (DoD)

Definición de “terminado”

- Conjunto de criterios que definen cuándo un **incremento de producto se considera completo y listo para su entrega** a los clientes.
- Entendimiento común entre todos los miembros del equipo.
- Tener criterios claros ayuda a enfocarse en la entrega de valor en cada sprint y a minimizar retrabajos.
- Toda la organización participa en la creación de la DoD, incluidos desarrolladores, testers, product owners y demás interesados.
- Se garantiza que todos utilicen la misma guía y checklist antes de marcar una tarea como finalizada

Definition of Done (DoD)

Ejemplos

Proyecto de desarrollo de software:

- Todo el código ha sido probado exhaustivamente mediante pruebas unitarias, de integración y de extremo a extremo.
- El incremento de producto se ha deployado en un entorno de pruebas (staging) y ha sido evaluado por el equipo.

En un proyecto genérico:

- Se han resuelto todos los errores.
- Se ha redactado y editado toda la documentación de la versión

Críticas a las Metodologías Ágiles

I don't need the documentation, I'll make it work



STOP DOING AGILE

- REQUIREMENTS ARE NOT SUPPOSED TO CHANGE EVERY TWO WEEKS
- YEARS of ESTIMATING yet no real-world evidence of anything getting done faster
- "In order to reduce process we add more process" — statement dreamed up by the UTTERLY DERANGED
- POKER is a GAME not a PLANNING TOOL
- T-shirt size estimation belongs in the CLOTHING STORE not SOFTWARE DEVELOPMENT
- "Points represent complexity, not time" YET YOU KEEP MEASURING HOW MANY POINTS FIT IN A WEEK??
- People who want to "BURN DOWN" belongs in JAIL not run software teams
- Same goes for people doing GROOMING

This is REAL SCRUM done by REAL scrum masters:



They have played us for absolute fools

¿Qué hacemos con la IA?

WATERFALL



AGILE



AI



Actividad Próxima clase

- Escuchar podcast “Inteligencia Artificial Semanal”, capítulo “Nada más práctico que una buena teoría: un ejemplo ilustrativo” del 24 de enero de 2026
- <https://tinyurl.com/Inge-Clase3>
- Es un capítulo out of context del podcast, a raíz de los accidentes de trenes en España en enero de 2026 (AVE: tren de alta velocidad)
- Pongan foco en lo que se plantea. No vamos a discutir la solución técnica, sino cómo se llega a ella
- Extraigan las “lecciones de vida”, más allá de la IA, el software y el producto